الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

Arab Academy for Science, Technology & Maritime Transport

BCD Instructions in Assembly Language

Overview

Binary-Coded Decimal (BCD) is a representation of decimal numbers in binary form. Each decimal digit is represented by a group of four bits (a nibble). For example:

- **BCD Number:** 0 9 D (0000–1001)
- **Structure:**
  - 1 byte = 8 bits
  - 1 nibble = 4 bits
  - Therefore, 1 byte = 2 nibbles

Types of BCD Representations

1. **Packed BCD:**
   - Each decimal digit is represented by 4 bits (1 nibble).
   - Example: $(28)_{10}$ → Packed BCD: 0010 1000
2. **Unpacked BCD:**
   - Each decimal digit is stored in a full byte (8 bits).
   - Example: $(28)_{10}$ → Unpacked BCD: 0000 0010 0000 1000

BCD (ASCII) Arithmetic

- The Intel Instruction set can handle both packed (two digits per byte) and unpacked BCD (one decimal digit per byte)
- We will first look at unpacked BCD
- Unpacked BCD can be either binary or ASCII. Consider the number 4567

  Bytes then look like 34h 35h 36h 37h

  OR: 04h 05h 06h 07h
- In packed BCD where there are two decimal digits per byte

  99h = 99d 4567d= 4567d

Unpacked BCD Arithmetic

- With unpacked BCD we wish to add strings such as '989' and '486' and come up with the string '1475'.
- With BCD you can use the standard input and output routines for strings to get numbers into and out of memory without converting to binary
- BCD arithmetic uses the standard binary arithmetic instructions and then converts the result to BCD using BCD adjustment instructions.

Where and Why is BCD used?

- BCD takes more space and more time than standard binary arithmetic
- It is used extensively in applications that deal with currency because floating point representations are inherently inexact
- Database management systems offer a variety of numeric storage options; "Decimal" means that numbers are stored internally either as BCD or as fixed-point integers
- BCD offers a relatively easy way to get around size limitations on integer arithmetic

From the SQL Server Manual

Exact Numerics	
bigint	numeric
bit	smallint
decimal	smallmoney
int	tinyint
money	
Approximate Numerics	
float	real

ASCII Values of Decimal Numbers

The ASCII representation of decimal numbers involves mapping each digit to a specific binary value.

For example:

ASCII	Binary	HEX	Symbol
65	`0100 0001`	41H	'A'
97	`0110 0001`	61H	'a'

For decimal digits `0–9`:

- ASCII values range from `48` to `57` (HEX: `30–39H`).

BCD Adjustment Instructions

- Four unpacked adjustment instructions are available:

AAA	(ASCII Adjust After Addition)
AAS	(ASCII Adjust After Subtraction)
AAM	(ASCII Adjust After Multiplication)
AAD	(ASCII ADjust BEFORE Division)

- Except for AAD the instructions are used to adjust results after performing a binary operation on ASCII or BCD data
- AAD (in spite of the mnemonic) is used after a DIV instruction

Packed BCD, ASCII, Unpacked BCD

- AAA and AAS can be used with both ASCII and unpacked BCD

9701 in ASCII (hex)	39 39 30 31
9701 in unpacked BCD	09 07 00 01

- Two Packed BCD operations are also available:

DAA	Decimal Adjust After Addition
DAS	Decimal Adjust After Subtraction

Key Instructions in Assembly Language

DAA (Decimal Adjust for Addition)

- **Purpose:** Adjusts the result of adding two packed BCD numbers to ensure a valid BCD representation.
- **Registers Affected:** Works only on the AL register. All flags are affected.
- **Algorithm:**
    1. If (D3–D0) > 9 or the Auxiliary Carry Flag (ACF) is set, add 06H to AL.
    2. If (D7–Dn) > 9 or the Carry Flag (CF) is set, add 60H to AL.

Example:

```assembly
MOV AL, num1        ; Load AL with 27H
ADD AL, num2        ; Add 35H to AL → Result: 5CH (92₁₀)
DAA                 ; Adjust AL → Result: 62H
```

Explanation:

- Initial `AL` = `5CH` (0101 1100)
- Low nibble = `C` (12) → Adjusted to `2` (12–10).
- High nibble incremented by 1 → `6`.
- Final result: `62` in packed BCD.

DAS (Decimal Adjust for Subtraction)

- **Purpose:** Similar to DAA but for subtraction. Converts the result of a subtraction operation to valid packed BCD.
- **Example:**

```assembly
MOV AL, 86H
SUB AL, 57H        ; AL = 2FH
DAS                ; Adjust AL → Result: 29H
```

Explanation:

- Initial `AL` = `2FH` (47 in decimal, invalid BCD).
- Subtract `6` from low nibble and adjust.
- Final result: `29` (valid packed BCD).

AAA (ASCII Adjust for Addition)

- **Purpose:** Converts the result of an ASCII-coded addition to unpacked BCD.
- **Flags Affected:** Updates Auxiliary Flag (AF) and Carry Flag (CF). Other flags are undefined.
- **Process:** Clears the higher nibble, keeps the lower nibble as a valid decimal, and adjusts the carry accordingly.

Example:

```assembly
MOV AL, 34H        ; ASCII for '4'
ADD AL, 38H        ; ASCII for '8' → Result: 6CH
AAA                ; Adjust AL → Result: 02H (unpacked BCD for 12)
```

AAS (ASCII Adjust for Subtraction)

- **Purpose:** Adjusts the result of subtracting ASCII-coded numbers to unpacked BCD format.
- **Example:**

```assembly
MOV AL, 39H       ; ASCII for '9'
SUB AL, 35H       ; ASCII for '5' → Result: 04H
AAS               ; Final result: 04H (unpacked BCD for 4)
```

AAM (ASCII Adjust for Multiplication)

- **Purpose:** Adjusts the result of multiplying two unpacked BCD numbers.
- **Registers Affected:** Converts the `AL` register's result into `AX`, splitting the quotient (`AH`) and remainder (`AL`).

Example:

```assembly
MOV AL, 09H       ; Unpacked BCD for 9
MOV BL, 05H       ; Unpacked BCD for 5
MUL BL            ; Multiply → Result: 002DH (45 in decimal)
AAM               ; Adjust → AX = 0405H (unpacked BCD for 45)
```

AAD (ASCII Adjust before Division)

- **Purpose:** Converts unpacked BCD digits in `AH` and `AL` into a packed BCD number, typically before division.
- **Example:**

```assembly
MOV AH, 04H       ; High byte
MOV AL, 03H       ; Low byte → AX = 0403H (43 in decimal)
AAD               ; Adjust → AX = 002BH (43 in packed BCD)
DIV BL            ; Divide AX by BL
```

Summary of Adjust Instructions

- **DAA/DAS:** Operate on packed BCD.
- **AAA/AAS/AAM/AAD:** Operate on unpacked BCD.
- Each instruction ensures results are in the appropriate BCD format for further processing or display.

Conclusion

The discussed instructions provide efficient methods to handle BCD arithmetic in assembly language, ensuring correct results in packed or unpacked formats. **DAA** and **DAS** are tailored for packed BCD adjustments during addition and subtraction, respectively. On the other hand, **AAA**, **AAS**, **AAM**, and **AAD** focus on unpacked BCD operations, aligning results for ASCII-coded arithmetic. These adjustments streamline BCD processing by managing carries, correcting nibbles, and converting formats. Mastering these instructions is essential for applications that require precise decimal computations, such as financial and embedded systems. Together, they demonstrate the flexibility of assembly language in low-level data manipulation.

References:

Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer Manuals*. [Online]. Available: https://www.intel.com.

https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html